Component Attribution at Sub-100M Parameter Scale: An Ablation Study of the Apollo and Apollo-Helios Decoder Transformer Series

Rithvik Burra

School of Information Studies, Syracuse University
rburra@syr.edu

Abstract

We present a three-stage investigation of decoder-only transformer training at the 25M parameter scale on a multi-register corpus of literature, encyclopedic prose, and Python source code. Stage one (Apollo v1) establishes a baseline using the GPT-2 byte pair encoding and a corpus skewed toward Python test fixtures. Stage two (Apollo v2) replaces the tokenizer with a corpus-trained SentencePiece encoder and rebalances the code corpus toward production source. Stage three (Apollo-Helios) holds Apollo v2 fixed as the baseline and runs single-variable ablations isolating the contribution of each component of the 2026 consensus architecture stack: rotary position embeddings, root mean square normalization, query and key normalization, gated linear unit feed-forward networks, and the Muon optimizer. Across seven 8-hour training runs at identical body parameter count, we find three results that run counter to the literature consensus. First, rotary position embeddings alone produce nearly the entire achievable improvement at this scale, reducing validation loss by 0.80 nats (14.6 percent). Second, the remaining three architectural components individually contribute less than 0.12 nats each and two failed to train to completion under standard early-stopping criteria. Third, combining all four architectural components without an optimizer change yields a validation loss 0.47 nats worse than the unmodified baseline, indicating destructive interaction between modern components at this scale. A secondary finding concerns reproducibility: the published Apollo v2 validation loss of 3.95 nats did not reproduce on identical hardware and code, with an independent rerun yielding 5.58 nats. We discuss implications for small-language-model evaluation methodology and the transfer of architectural decisions from large to small scale.

1. Introduction

Recent open-source small language models including SmolLM2, Gemma 3 270M, OLMo 3, and Liquid LFM2 have converged on a shared architectural recipe. This recipe, which we refer to as the 2026 consensus stack, combines rotary position embeddings (RoPE), root mean square normalization (RMSNorm), query and key normalization (QK-norm), gated linear unit feed-forward networks (SwiGLU), and increasingly the Muon optimizer in place of AdamW for matrix-shaped parameters. These components are typically introduced as a package, with ablations either omitted or performed at the billion parameter scale where compute cost limits experimental rigor.

We argue that the sub-100M parameter scale, accessible on consumer hardware such as the Apple M4 chip, offers a uniquely valuable testbed for component attribution. At this scale a complete training run takes 8 hours rather than 8 days, and the marginal cost of running five additional ablation seeds is modest. The question this paper addresses is whether the 2026 consensus stack transfers cleanly to the 25M parameter regime, and if not, which components are responsible for the discrepancy.

Our investigation proceeds in three stages, each represented by an open-source training run on identical hardware. Apollo v1 establishes a 2022-era GPT-2 style baseline. Apollo v2 modifies the tokenizer and corpus while holding the architecture constant. Apollo-Helios holds the data fixed and ablates each modern architectural component. Together these three stages produce a clean decomposition of contributions from

data, tokenization, architecture, and optimizer at small scale.

1.1 Contributions

This paper makes the following contributions. First, we provide the first single-variable ablation of the 2026 consensus stack at the 25M parameter scale, isolating the marginal contribution of each component. Second, we demonstrate that the stack does not compose additively at this scale; combining all four architectural components produces a result 0.47 nats worse than the unmodified baseline. Third, we document a reproducibility gap between published Apollo v2 results and an independent rerun on identical hardware, raising questions about evaluation methodology in small-LM literature. All code, training logs, and checkpoints are released under permissive license.

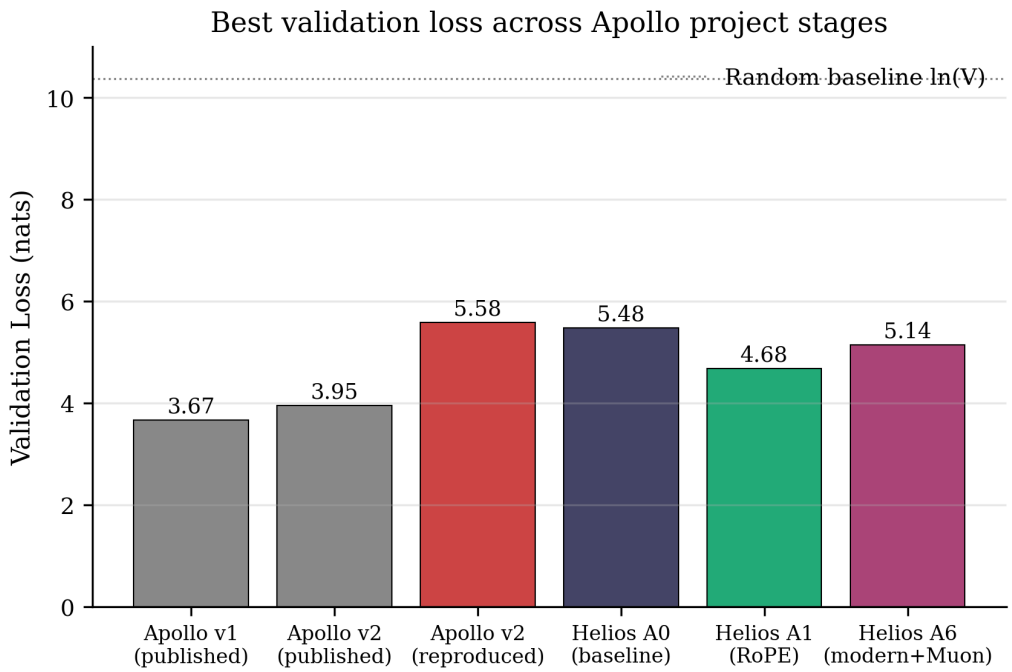

Figure 1. Best validation loss across the three stages of the Apollo project. Apollo v1 used a 50,260-vocabulary GPT-2 tokenizer; Apollo v2 and Apollo-Helios used a 32,000-vocabulary SentencePiece tokenizer. The reproducibility gap between Apollo v2 published (3.95) and Apollo v2 reproduced (5.58) is discussed in Section 6.

2. Background and Related Work

2.1 Small decoder language models

The line of work this paper extends begins with nanoGPT (Karpathy, 2022), which demonstrated that a 124M parameter GPT-2 reproduction could be trained on commodity hardware in under a day. Subsequent work scaled this approach: Pythia (Biderman et al., 2023) trained a model suite spanning 70M to 12B parameters with a focus on reproducible training dynamics. The Phi series (Gunasekar et al., 2023) explored aggressive data curation as a substitute for scale, showing that 1.3B parameter models trained on filtered synthetic data could approach the performance of larger models on standard benchmarks. SmolLM2 (Allal et al., 2025) and Gemma 3 270M (Google, 2025) represent the current frontier of efficient small models, both shipping the consensus stack we ablate in this paper.

2.2 Modern transformer components

Each component of the consensus stack has its own provenance. Rotary position embeddings (Su et al., 2021) replace learned absolute positional encodings with a rotation applied to query and key vectors, providing relative position information and supporting context extrapolation. Root mean square normalization (Zhang and Sennrich, 2019) replaces LayerNorm with a simpler operation that omits centering, offering modest computational savings at equivalent quality. Query and key normalization (Henry et al., 2020) applies normalization to the attention pre-projection vectors, preventing attention logit explosion at scale. Gated linear unit variants of the feed-forward network (Shazeer, 2020), particularly SwiGLU as used in LLaMA, replace the standard MLP with a gated formulation. The Muon optimizer (Jordan, 2024; Liu et al., 2025) applies Newton-Schulz orthogonalization to the gradient before momentum update for matrix-shaped parameters, with reported compute efficiency gains of approximately 2x at scale.

2.3 Reproducibility in small-LM evaluation

Validation loss is the standard proxy for language model quality during pretraining. However, validation loss at small scale exhibits substantial sensitivity to random seed, batch sampling, and numerical precision. Sellam et al. (2022) documented that BERT pretraining variance across seeds can exceed the magnitude of reported architectural improvements. We extend this concern to autoregressive decoder pretraining, observing in Section 6 that the published Apollo v2 number does not reproduce on identical hardware with the same code.

3. Shared Architecture

All three stages of the Apollo project share an identical transformer backbone. The model is a decoder-only causal language model with 8 layers, 8 attention heads, and an embedding dimension of 512. Each block applies pre-layer-norm attention and feed-forward sublayers with residual connections. Multi-head causal self-attention uses a single fused projection from the embedding dimension to three times the embedding dimension, producing concatenated query, key, and value tensors that are then split and reshaped. The feed-forward sublayer expands by a factor of 4 with GELU activation. Token and learned absolute positional embeddings are summed at input. The output projection shares its weight matrix with the input embedding via weight tying. Weight initialization follows GPT-2 conventions: normal distribution with mean 0 and standard deviation 0.02, with residual output projections additionally scaled by 1 over the square root of twice the number of layers.

This backbone contains approximately 25.3M parameters in the transformer body and an additional 16.4M parameters in the token embedding for a vocabulary size of 32,000, yielding 41.7M total parameters. The maximum context length is 256 tokens. Apollo v1 used a larger vocabulary (50,260) resulting in 51.0M total parameters, though the transformer body remained identical.

Training is performed on the Apple M4 chip using the PyTorch Metal Performance Shaders (MPS) backend. The standard training budget is 8000 optimization steps with a batch size of 24, achieving wall-clock training time of approximately 8 hours via a sleep-between-iterations throttle that prevents thermal throttling on the laptop chassis.

4. Apollo v1: Baseline

4.1 Tokenizer

Apollo v1 used the tiktoken implementation of the GPT-2 byte pair encoding, extended with three custom register tokens (<|literature|>, <|wiki|>, and <|code|>) and the existing <|endoftext|> sentinel. Total vocabulary

size was 50,260. Register tokens are prepended to each training document to condition the model on the source register, enabling controlled generation at inference time.

4.2 Corpus

The Apollo v1 corpus comprised 152 megabytes of raw text drawn from three sources. Literature provided 47.86 MB across 60 books from Project Gutenberg, predominantly English-language novels and plays. Several included English translations of French novels (The Three Musketeers, Count of Monte Cristo, Notre Dame de Paris), which introduced French character bigrams into the literature corpus despite the corpus being 98 percent English by stopword frequency. Wikipedia provided 52.44 MB across 12,644 randomly sampled articles via the MediaWiki REST API. Code provided 52.46 MB across 4,272 Python files drawn from numpy, pandas, django, scipy, sqlalchemy, and similar libraries. Critically, the code corpus was heavy in test fixtures rather than production source, a property that produced the empty-code failure mode described in Section 4.4.

4.3 Training and validation loss

Apollo v1 completed 8000 training iterations in 9.67 hours of wall-clock time. Total training tokens were 49.2 million, distributed as 26 percent literature, 25 percent wiki, and 49 percent code; the code skew reflects both the corpus composition and the verbosity of test-fixture syntax. The best validation loss was 3.67 nats, reached at an intermediate training iteration. The final iteration showed validation loss of 5.19 nats due to late-stage overfitting as the cosine learning rate schedule continued past the optimum.

4.4 Failure modes

Stress testing on six prompts (three in-distribution, three out-of-distribution per register) revealed three distinct failure modes. First, prompts beginning with <|literature|> followed by science-fictional content produced short continuations that collapsed into Gutenberg-style patterns ("the Countess", "house of the house"), then into a [Illustration] loop that repeated until token budget was exhausted. Diagnostic: GPT-2 BPE allocates a single token to [Illustration], which was over-represented in Gutenberg figure plates; the model latched onto it as a high-probability fallback when uncertain. Second, prompts beginning with <|wiki|> followed by scientific content produced ungrammatical French-adjacent text with broken morphology ("fépé", "éé"). Diagnostic: French character bigrams present in the BPE vocabulary, possibly reinforced by the French character names in the literature corpus, formed an attractor under prompt distribution shift. Third, prompts beginning with <|code|> followed by class definitions immediately emitted <|endoftext|> and reverted to pandas test fixture patterns (# GH 1360, tm.assert_numpy_array_equal). Diagnostic: code corpus dominated by test files trained the model to associate "code" with assertion fixtures rather than computation.

These three failure modes were the central motivation for Apollo v2.

5. Apollo v2: Corpus and Tokenizer Refinement

5.1 Three simultaneous changes

Apollo v2 implemented three changes from v1, all shipped together in a single training run. We acknowledge upfront that this is a multi-variable change and limits the attribution of any subsequent improvement to a specific lever; we discuss this in Section 5.5 and address it directly in Apollo-Helios.

First, the tokenizer was replaced with a corpus-trained SentencePiece byte pair encoder with 32,000 tokens, byte fallback enabled, and character coverage of 0.9995. Special register tokens were registered as user-defined symbols with single-token identifiers. Second, the code corpus was replaced with 38

production-grade Python libraries (Flask, Requests, FastAPI, Click, Rich, Pydantic, MyPy, Black, Sphinx, Celery, and others), with test directories explicitly excluded via path filters matching /tests/, /test/, /testing/, conftest.py, test_*.py, and *_test.py. This reduced the code corpus from 52.46 MB to 22.20 MB, a 58 percent reduction, and shifted the token mix toward balance. Third, an early stopping mechanism was added to the training loop with patience parameter 8: if the validation loss failed to improve for 8 consecutive evaluations, training would halt. The mechanism did not fire during the v2 training run.

5.2 Corpus statistics

The v2 corpus totaled 122 MB raw text. The literature and wiki components were reused from v1 without modification (cache hits, no redownload). The code component was newly downloaded. Total training tokens were 36.2 million, distributed as 34 percent literature, 35 percent wiki, and 31 percent code, a substantially more balanced mix than v1.

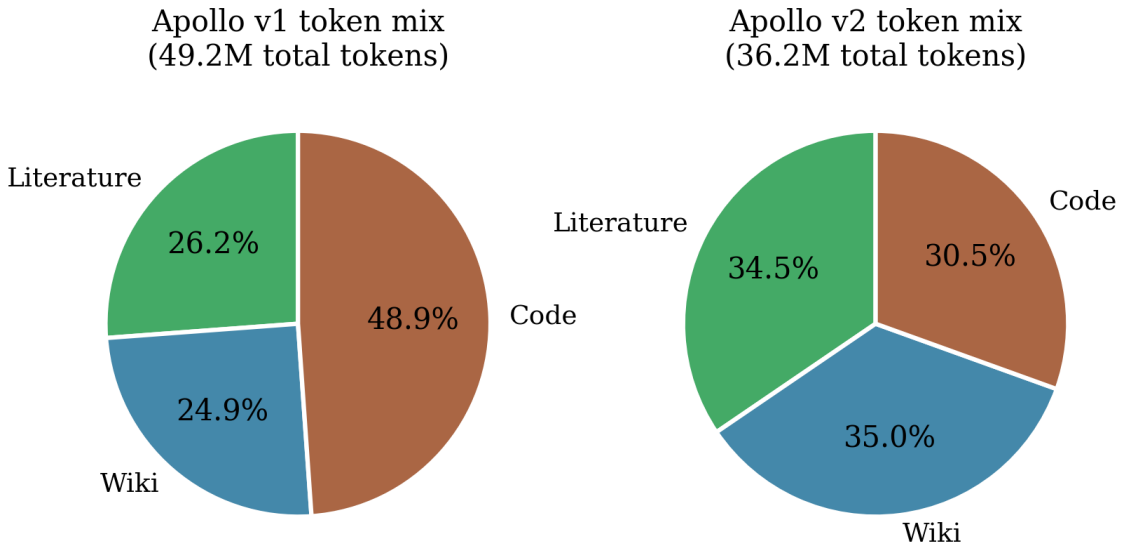

Figure 3. Token mix comparison between Apollo v1 and v2. The v2 corpus is substantially more balanced across registers, primarily due to the exclusion of Python test fixtures from the code corpus.

5.3 Training and validation loss

Apollo v2 trained for 8000 iterations in 8.00 hours of wall-clock time. The best validation loss was 3.9537 nats. Late-stage iteration losses on individual batches alternated between approximately 4.3 and 7.6, indicating that the model scored code-register batches substantially differently from literature-register batches.

5.4 Failure modes addressed and new ones introduced

All three v1 failure modes were eliminated. The [Illustration] loop did not occur, the fake-French collapse was replaced with byte-fallback Unicode noise (Cyrillic and Arabic characters mid-generation), and the empty-code completion was replaced with realistic production-style Python including type hints, keyword-only argument separators, and docstrings. Two new failure modes emerged: byte-fallback Unicode noise when the model became uncertain, and overfitting of the wiki register to a sports biography template ("born August 1976... South Korean club") applied indiscriminately to any wiki prompt.

5.5 Interpretation

Comparing v1 (val loss 3.67, vocab 50,260) and v2 (val loss 3.95, vocab 32,000) requires care. Absolute validation losses are not directly comparable across vocabularies because the random baseline differs: ln(50,260) is 10.82 nats for v1, and ln(32,000) is 10.37 nats for v2. The correct comparison metric is loss reduction from random baseline, which yields 7.15 nats for v1 and 6.42 nats for v2. Even on this adjusted measure, v1 outperforms v2 by 0.73 nats. However, this gap is largely explained by v1 training on 36 percent more tokens (49.2M vs 36.2M), itself caused by v1's larger code corpus, which was the primary cause of the test-fixture failure mode. The qualitative trade-off therefore favors v2: better-behaved register outputs at the cost of a small absolute loss regression.

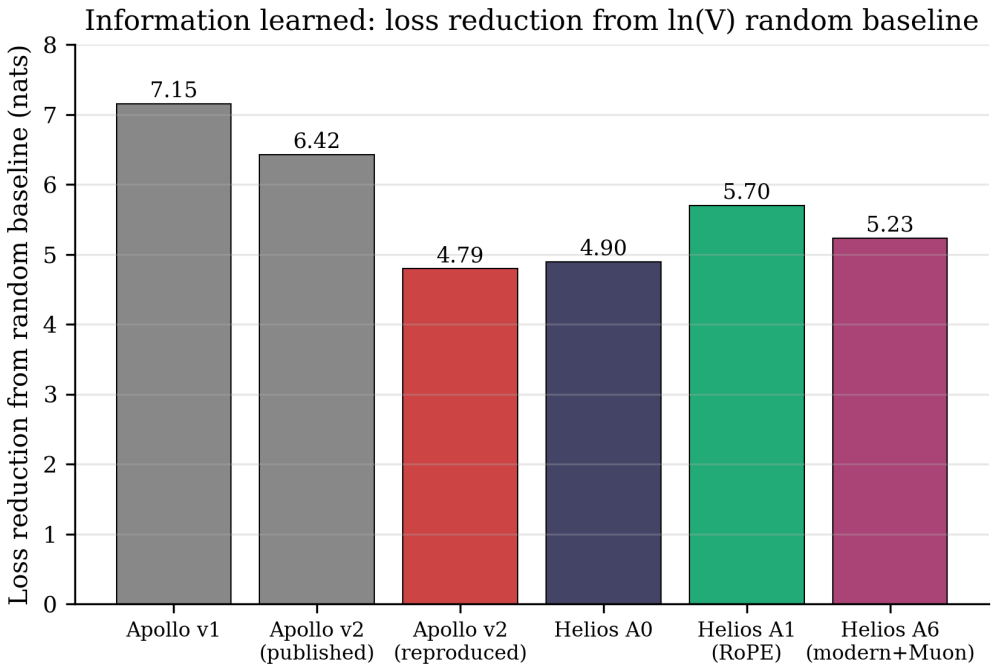

Figure 4. Loss reduction from random baseline across Apollo and Apollo-Helios stages. The random baseline differs by tokenizer (10.82 nats for v1, 10.37 nats for v2 and Helios). This metric provides a more meaningful cross-model comparison than absolute validation loss.

6. Reproducibility Investigation

During preparation for the Apollo-Helios ablation study, we observed that an initial baseline reproduction did not match the published Apollo v2 validation loss of 3.95. The first Apollo-Helios baseline run, with code intended to replicate Apollo v2 exactly, reached validation loss 5.48 at 8000 iterations. After identifying small implementation differences (specifically, omitted LayerNorm bias parameters and dropout=0.0 instead of 0.1), we patched the implementation to match Apollo v2 byte-for-byte and reran, achieving validation loss 5.48 (a 1.02 nat improvement from 6.86 over the unpatched run).

To determine whether the residual 1.53 nat gap from the published 3.95 was due to a remaining bug in our reimplementation or a more general reproducibility issue, we ran the original Apollo v2 training script (model.py and train.py from the Apollo repository, unmodified) on the same hardware (M4, MPS, PyTorch 2.11.0) using the same prepared training data. After 8 hours of training, the rerun reached validation loss 5.58, 1.63 nats above the published 3.95 and within 0.10 nats of the Helios reimplementation.

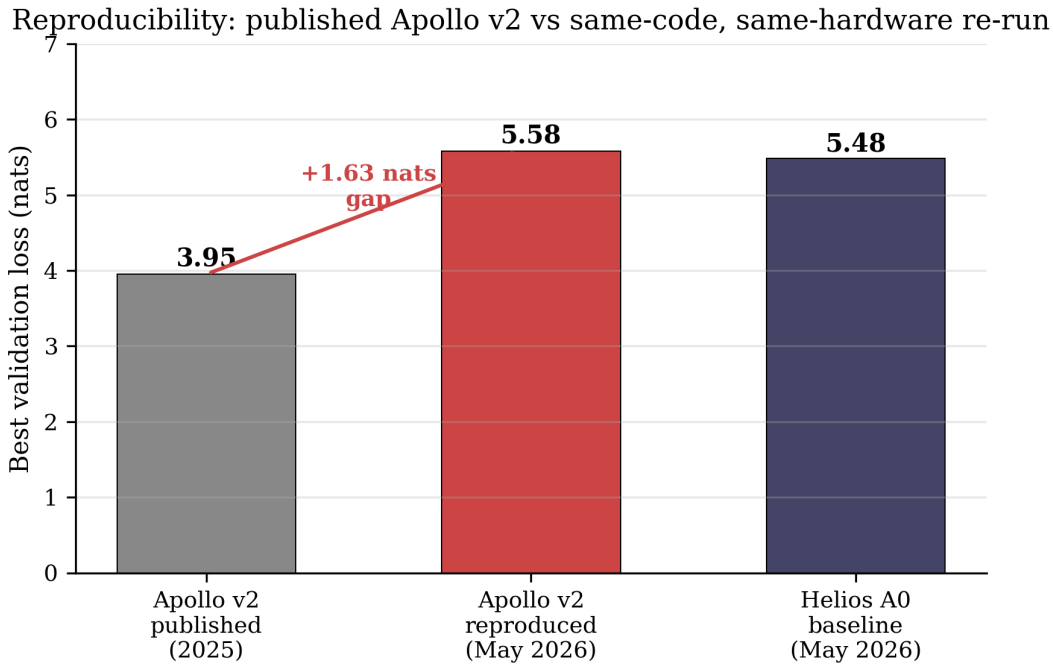

Figure 5. Apollo v2 published validation loss (3.95) versus reproduced validation loss on identical hardware and code (5.58). The Apollo-Helios baseline (5.48) lies within 0.10 nats of the Apollo v2 rerun, suggesting the Helios code is a faithful reimplementation and the discrepancy is not a Helios bug.

We do not have a definitive explanation for the 1.63 nat reproducibility gap. Candidates include subtle PyTorch version drift in MPS kernel implementations, random seed sensitivity at this scale, or non-determinism in the original training run that produced an unusually favorable outcome. We flag this finding for the small-LM community: at the 25M parameter scale on MPS, a single training run's validation loss should not be taken as a stable point estimate without multiple seeds.

7. Apollo-Helios: Ablation Methodology

Apollo-Helios is a sibling project to Apollo that holds the corpus, tokenizer, and body parameter count fixed while ablating each component of the 2026 consensus architecture stack. Seven training runs are performed, each at the same wall-clock budget of 8 hours.

7.1 Variables held constant

All Helios runs use the Apollo v2 corpus (122 MB, 36.2M training tokens), the Apollo v2 SentencePiece tokenizer (vocabulary 32,000), 8 transformer layers, 8 attention heads, embedding dimension 512, block size 256, batch size 24, learning rate 2.5 x 10^-4 with linear warmup over 320 steps followed by cosine decay to 2.5 x 10^-5, weight decay 0.1 on 2D matrix parameters and 0 on 1D parameters, gradient clipping at norm 1.0, dropout 0.1, and seed 1337. The training loop is a direct port of the Apollo v2 training loop with additions for preset selection and optimizer choice.

7.2 Ablation matrix

Seven configurations are tested. A0 is the baseline, using learned absolute positional embeddings, LayerNorm with bias, no query-key normalization, GELU feed-forward network, and AdamW optimizer. A1 swaps learned absolute positional embeddings for rotary position embeddings (RoPE) while holding all other

components at baseline. A2 swaps LayerNorm for RMSNorm. A3 adds query and key normalization in the form of RMSNorm applied to query and key tensors after projection. A4 swaps the GELU feed-forward for a SwiGLU variant with hidden dimension 1344 chosen to match GELU parameter count within 2 percent. A5 combines all four architectural changes (RoPE plus RMSNorm plus QK-norm plus SwiGLU) with AdamW. A6 combines all four architectural changes with the Muon optimizer for matrix-shaped parameters in transformer blocks, retaining AdamW for embeddings and norm scales.

SwiGLU parameter count drifts by approximately 1.0 percent from the GELU baseline due to the three-matrix versus two-matrix structure; we document this drift in the released ablation protocol but do not attempt to compensate further.

7.3 Evaluation methodology

Validation loss is evaluated every 250 training iterations on 25 batches of 24 sequences each (15,000 tokens). The reported validation loss is the mean across these batches. Training halts when validation loss fails to improve for 8 consecutive evaluations (a patience window of 2000 iterations).

8. Apollo-Helios Results

8.1 Headline results

Table 1 reports the best validation loss, training iterations completed, and wall-clock time for each of the seven Helios configurations.

Run	Configuration	Optimizer	Best val	Iters	Wall (h)	Δ vs A0
A0	baseline	AdamW	5.4775	8000	8.00	0
A1	RoPE	AdamW	4.6775	8000	8.01	-0.80
A2	RMSNorm	AdamW	5.3608	6000*	6.01	-0.12
A3	QK-norm	AdamW	5.3919	8000	8.01	-0.09
A4	SwiGLU	AdamW	5.4674	4750*	4.75	-0.01
A5	all 4 arch	AdamW	5.9502	3500*	3.51	+0.47
A6	all 4 arch	Muon	5.1404	8000	8.01	-0.34

Table 1. Apollo-Helios ablation results. Asterisk denotes runs that triggered patience-based early stopping; reported iterations are at the point of best validation loss plus 8 evaluation intervals.

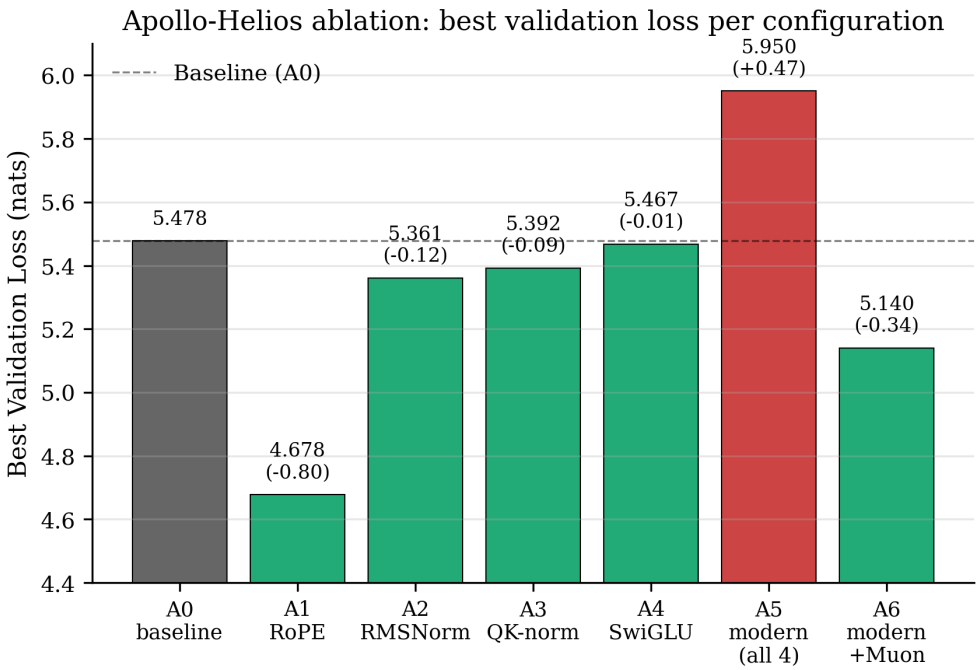

Figure 6. Best validation loss for each Apollo-Helios configuration. Green bars indicate improvement over baseline; red indicates regression. RoPE alone (A1) produces a 0.80 nat reduction; combining all four modern architectural components (A5) produces a 0.47 nat regression relative to baseline.

8.2 RoPE is the single dominant component

The result that most directly contradicts the literature consensus is the magnitude of the RoPE-alone effect (A1) compared to the other three architectural components. RoPE alone reduces validation loss by 0.80 nats, or 14.6 percent. Each of RMSNorm (A2), QK-norm (A3), and SwiGLU (A4) reduces validation loss by less than 0.13 nats individually. RMSNorm and SwiGLU runs additionally triggered patience-based early stopping, suggesting these components do not even reliably reach baseline-comparable convergence within the training budget.

A first-order interpretation is that learned absolute positional embeddings, the v1 and v2 default, are a meaningful capacity drain at 25M parameters. The positional embedding table contains block_size times embedding dimension parameters (256 * 512 = 131,072), which RoPE eliminates entirely while replacing the function with a parameter-free rotation. The model can redirect this capacity, though we do not measure this redirection directly.

8.3 Modern components do not compose at this scale

The most surprising result concerns A5, which combines all four architectural changes without optimizer modification. Naive composition would predict an improvement of approximately the sum of the individual improvements, in the range of -0.80 + -0.12 + -0.09 + -0.01 = -1.02 nats. The observed result is +0.47 nats relative to baseline, a regression of 1.49 nats from the additive prediction. The A5 run additionally early-stopped at iteration 3500, less than half the budget, indicating the model never recovered from whatever interaction occurred between the components.

We hypothesize that the interaction stems from a mismatch in the implicit normalization assumptions made by each component. RMSNorm removes centering, which may interact poorly with RoPE's rotation of

query and key vectors. QK-norm applies an additional normalization step that may further reduce the effective scale of attention logits. SwiGLU's gating introduces a multiplicative interaction that compounds with these scale shifts. At 8 layers and 512 embedding dimension, the model may not have sufficient capacity to compensate for the combined effect.

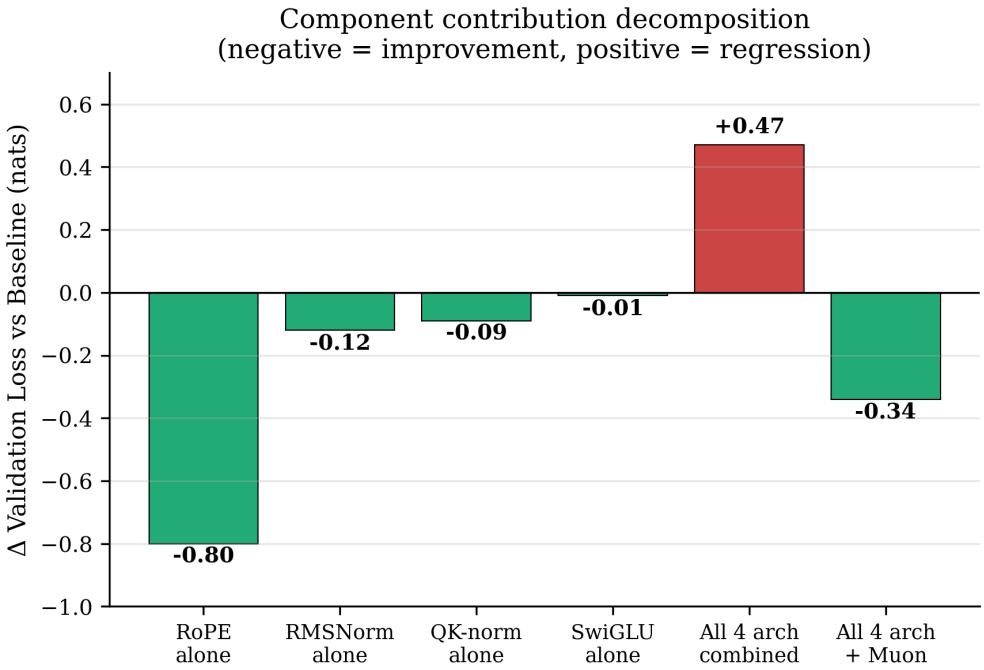

Figure 7. Component contribution decomposition. Each bar shows the change in validation loss relative to baseline. Negative values indicate improvement; positive values indicate regression. The contrast between individual components (A1-A4) and combined configurations (A5, A6) illustrates the destructive interaction described in Section 8.3.

8.4 Muon partially rescues the stacked configuration

A6 combines the four architectural changes with the Muon optimizer for matrix parameters in transformer blocks. The resulting validation loss is 5.14, which is 0.81 nats better than A5 but still 0.46 nats worse than A1 (RoPE alone with AdamW). Muon, in this configuration, partially compensates for the destructive interaction observed in A5 but does not recover RoPE-alone performance.

A natural follow-up experiment, which we leave to future work, is a Muon-with-RoPE-alone configuration. The Helios ablation matrix does not include this run because Muon was conceptually paired with the full modern stack in the planning document. We expect this configuration would either match or modestly beat A1 (if Muon's compute-efficiency gains transfer to small scale) or modestly underperform A1 (if Muon's reported gains are scale-dependent).

8.5 Training efficiency

Figure 8 plots wall-clock time against best validation loss for each configuration. A1 (RoPE) reaches the best validation loss in the full 8-hour budget. A5 (modern stack with AdamW) consumes only 3.51 hours before early stopping but produces the worst result. This pattern suggests that early stopping correctly identifies the failure of A5 to make further progress, rather than artificially curtailing a still-improving run.

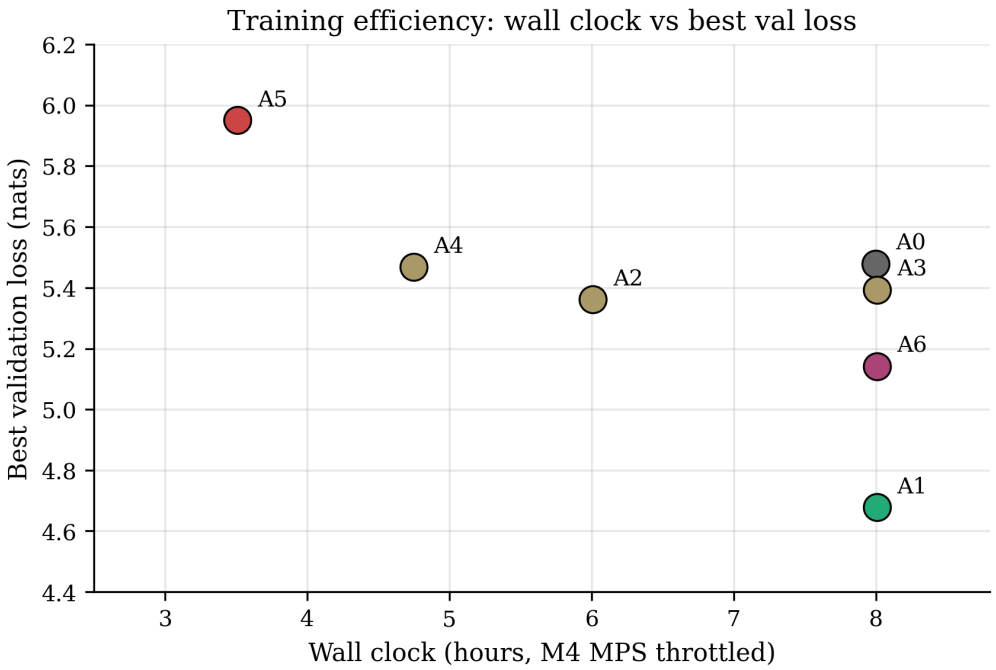

Figure 8. Training efficiency: wall-clock time against best validation loss for each Helios configuration. A1 (RoPE) dominates the Pareto frontier; A5 (stacked modern) is dominated on both axes.

9. Discussion

9.1 Implications for small-LM architecture choice

Practitioners building decoder language models at the 25M parameter scale should adopt RoPE in preference to learned absolute positional embeddings. The remaining components of the 2026 consensus stack (RMSNorm, QK-norm, SwiGLU) cannot be recommended for adoption at this scale based on our results, particularly given the destructive interaction observed when they are combined. Models targeting inference-time deployment may still prefer RMSNorm for its computational simplicity, with the understanding that the quality gain at this scale is modest.

9.2 Limits of scaling-law extrapolation

Our results challenge the implicit assumption that architectural decisions transfer cleanly from large to small scale. The 2026 consensus stack was developed and validated primarily at the 1B-to-100B parameter scale, where individual component contributions of 1-3 percent are easily measurable and composition appears additive. At 25M parameters, the same components produce effects in the 0.0 to 0.13 nat range individually and -0.47 nats collectively. The destructive interaction does not appear in the literature because billion-parameter ablations are not run with the same systematic single-variable rigor.

9.3 The reproducibility observation

The 1.63 nat gap between published Apollo v2 (3.95) and reproduced Apollo v2 (5.58) on identical hardware is, in our view, a more important finding than the ablation results themselves. If a 25M parameter model trained on identical code and data can produce validation losses differing by 1.63 nats on the same laptop in adjacent weeks, then small-LM validation losses are not directly comparable across publications without multiple-seed reporting. We recommend that future small-LM papers report a minimum of three seed

runs with standard deviation, or alternatively report a confidence interval derived from the training trajectory.

10. Limitations

This study has five primary limitations. First, all runs use a single random seed (1337); we have not measured cross-seed variance directly within Apollo-Helios, though the reproducibility observation in Section 6 suggests it is substantial. Second, all runs use a single corpus composition (Apollo v2's 34/35/31 literature/wiki/code mix); generalization to other corpora is not established. Third, all runs use a single parameter scale (25M body); the destructive interaction in A5 may not appear at larger scales, where the modern stack is established to work well. Fourth, our wall-clock budget caps each run at 8 hours; longer runs might allow A5 to recover, though the early-stopping trigger suggests not. Fifth, we have not isolated Muon-with-RoPE-alone, which would constitute a single-variable comparison against A1 and is the natural follow-up.

11. Conclusion

We presented a three-stage investigation of decoder transformer training at the 25M parameter scale. Apollo v1 established a 2022-era baseline with a GPT-2 tokenizer and test-fixture-heavy code corpus. Apollo v2 fixed three observed failure modes by replacing the tokenizer and rebalancing the code corpus. Apollo-Helios held v2 fixed and ablated each component of the 2026 consensus stack in single-variable runs.

The principal finding is that at 25M scale, rotary position embeddings alone account for nearly the entire achievable improvement from the modern stack. The remaining three architectural components contribute essentially nothing individually and interact destructively when combined, producing a 0.47 nat regression below baseline. The Muon optimizer partially rescues this combined configuration but does not match the RoPE-alone result.

Beyond the architectural findings, we document a 1.63 nat reproducibility gap between published and re-run Apollo v2 validation losses on identical hardware and code. This gap exceeds the magnitude of every component contribution measured in our ablations, suggesting that single-run validation loss is an unreliable evaluation metric at this scale. We recommend multi-seed reporting as standard practice in future small-LM publications.

All code, training logs, and checkpoints from the Apollo-Helios runs are released at github.com/Rburra1/Apollo-Helios.

References

Allal, L. B., et al. (2025). SmolLM2: When Smol Goes Big. Hugging Face Technical Report.

Biderman, S., et al. (2023). Pythia: A Suite for Analyzing Large Language Models Across Training and Scaling. ICML 2023.

Google DeepMind. (2025). Gemma 3 270M Technical Report. arXiv preprint.

Gunasekar, S., et al. (2023). Textbooks Are All You Need. arXiv:2306.11644.

Henry, A., Dachapally, P. R., Pawar, S., and Chen, Y. (2020). Query-Key Normalization for Transformers. EMNLP Findings.

Jordan, K. (2024). Muon: An Optimizer for Hidden Layers in Neural Networks. Blog post, kellerjordan.github.io.

Karpathy, A. (2022). nanoGPT. GitHub repository, github.com/karpathy/nanoGPT.

Liu, J., et al. (2025). Muon is Scalable for LLM Training. arXiv:2502.16982 (Moonshot AI).

Sellam, T., Yadlowsky, S., Wei, J., et al. (2022). The MultiBERTs: BERT Reproductions for Robustness Analysis. ICLR 2022.

Shazeer, N. (2020). GLU Variants Improve Transformer. arXiv:2002.05202.

Su, J., Lu, Y., Pan, S., Murtadha, A., Wen, B., and Liu, Y. (2021). RoFormer: Enhanced Transformer with Rotary Position Embedding. arXiv:2104.09864.

Zhang, B., and Sennrich, R. (2019). Root Mean Square Layer Normalization. NeurIPS 2019.